

1. Question

Develop a “Library Management System” to demonstrate Object-Oriented Programming concepts.

Part A: Classes and Objects

Create a class named Book containing:

Book ID

Book Name

Author

Price

Create objects and display their details.

Part B: Constructors and Encapsulation

Create both default and parameterized constructors.

Declare data members as private.

Access them using Getter and Setter methods.

Part C: Inheritance

Create the following classes:

Person

Student — inherits Person

Faculty — inherits Person

Display the inherited properties using objects.

Part D: Polymorphism

Implement:

Method Overloading for calculating the area of different shapes.

Method Overriding by creating a Vehicle class and overriding the display() method in Car and Bike classes.

Part E: Abstraction and Interfaces

Create an abstract class Shape containing an abstract method draw().

Implement the draw() method in Circle and Rectangle.

Create an interface Printable and implement it in a class named Report.

2. Steps for Execution

Install Java JDK on the system.

Open Eclipse, IntelliJ IDEA, NetBeans, VS Code, or another Java IDE.

Create a Java file named:

LibraryManagementSystem.java

Enter the given Java program.

Save the program.

Compile the program:

```
javac LibraryManagementSystem.java
```

Run the program:

```
java LibraryManagementSystem
```

The program creates a Book object using the default constructor.

Setter methods are used to assign book details.

The details of the first book are displayed.

A second Book object is created using the parameterized constructor.

Student and Faculty objects are created to demonstrate inheritance.

The Area object demonstrates method overloading.

Vehicle, Car, and Bike objects demonstrate method overriding and runtime polymorphism.

Shape, Circle, and Rectangle demonstrate abstraction.

Report implements the Printable interface.

All results are displayed on the console.

3. Algorithm

Step 1

Start the program.

Step 2

Create the Book class with private data members:

bookId

bookName

author

price

Step 3

Create a default constructor and a parameterized constructor for Book.

Step 4

Create getter and setter methods to demonstrate encapsulation.

Step 5

Create a Book object using the default constructor.

Step 6

Set its details using setter methods and display the details.

Step 7

Create another Book object using the parameterized constructor and display its details.

Step 8

Create the Person class.

Step 9

Create Student and Faculty classes that inherit from Person.

Step 10

Create Student and Faculty objects and display their inherited properties.

Step 11

Create the Area class with overloaded area() methods for:

Square

Rectangle

Circle

Step 12

Create an Area object and call the overloaded methods.

Step 13

Create the Vehicle class with a display() method.

Step 14

Create Car and Bike classes that override the display() method.

Step 15

Use a Vehicle reference to refer to Car and Bike objects and call display().

Step 16

Create an abstract Shape class containing the abstract draw() method.

Step 17

Create Circle and Rectangle classes extending Shape and implement draw().

Step 18

Create the Printable interface containing print().

Step 19

Create the Report class implementing Printable.

Step 20

Create a Report object and call the print() method.

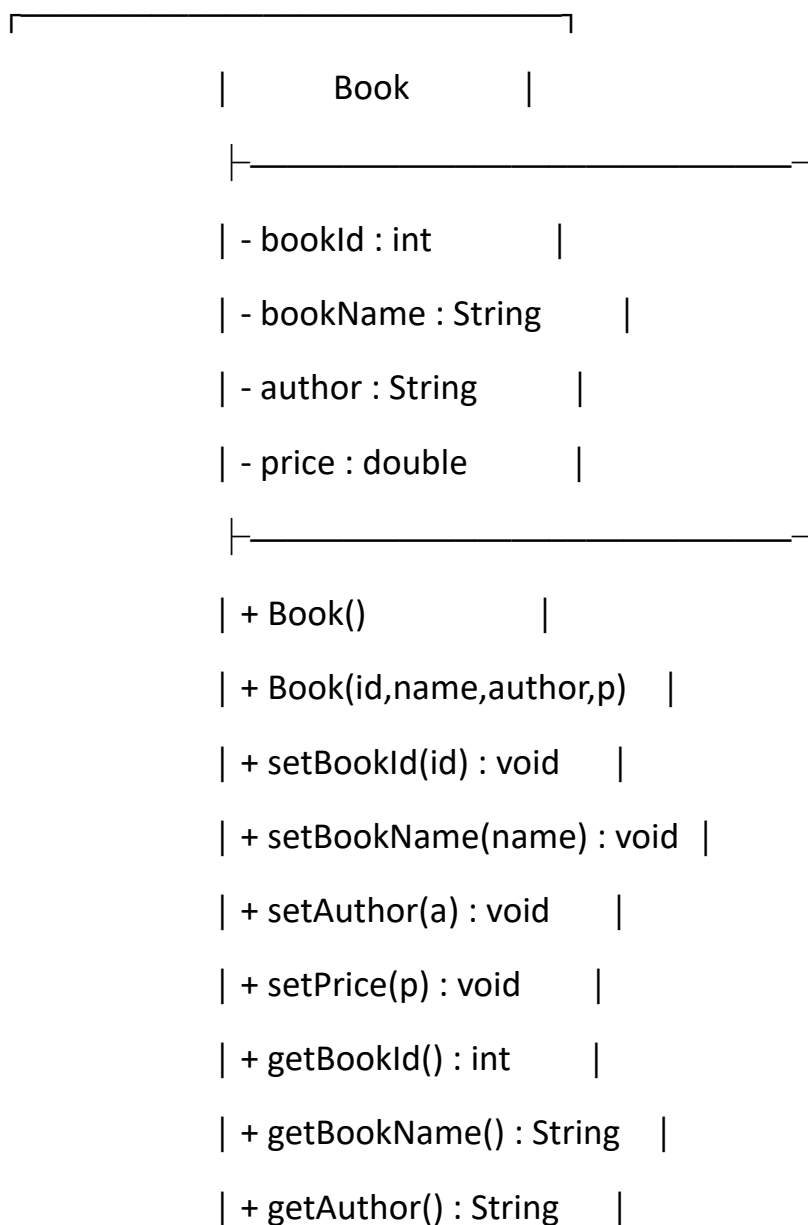
Step 21

Display all results.

Step 22

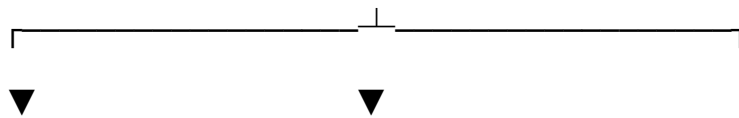
Stop the program.

UML CLASS DIAGRAM :



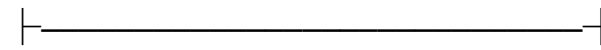
```
| + getPrice() : double |
| + display() : void    |
└────────────────────────┘
```

```
┌────────────────────────┐
|      Person      |
└────────────────────────┘
| name : String    |
| age : int        |
└──────────┴──────────┘
|
|
```



```
┌──────────┐      ┌──────────┐
| Student  |      | Faculty  |
└──────────┘      └──────────┘
| rollNo : int |    | subject : String |
└──────────┘      └──────────┘
| displayStudent() | | displayFaculty() |
└──────────┘      └──────────┘
```

```
┌────────────────────────┐
|      Area      |
└────────────────────────┘
```



| + area(side : int) : void |

| + area(l,b : int) : void |

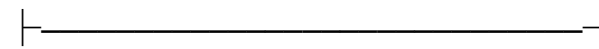
| + area(r : double) : void |



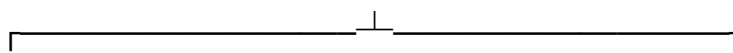
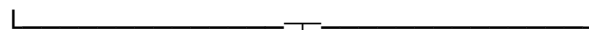
Method Overloading



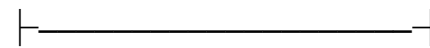
| Vehicle |



| + display() : void |



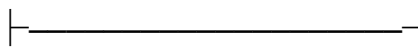
| Car |



| + display() : void |

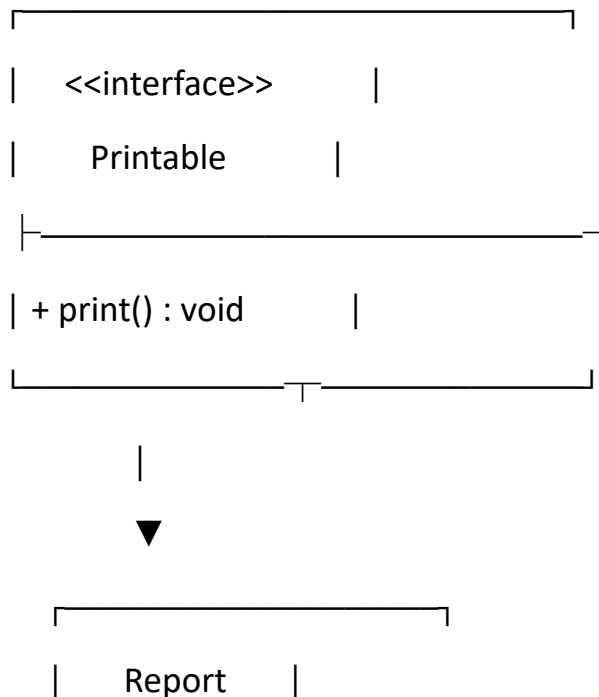
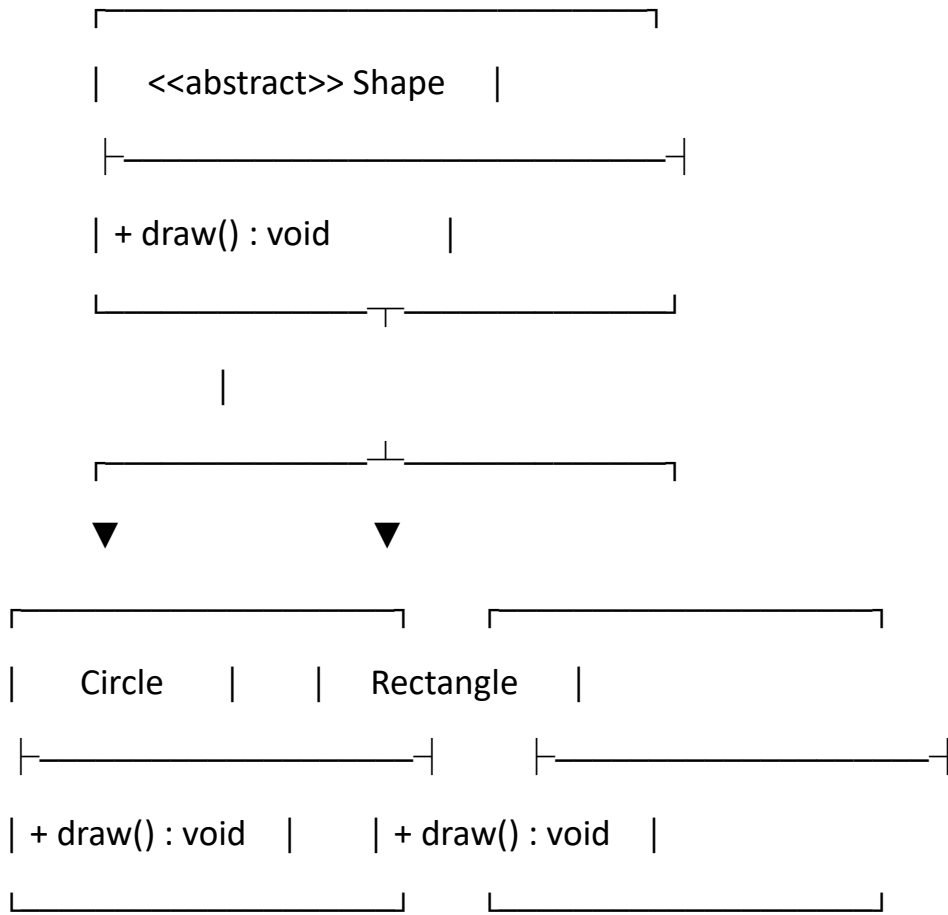


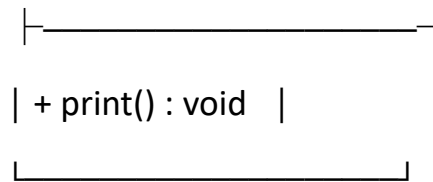
| Bike |



| + display() : void |







CODE:

```
class Book {  
    private int bookId;  
    private String bookName;  
    private String author;  
    private double price;  
    public Book() {  
        System.out.println("Default Constructor Called");  
    }  
    public Book(int id, String name, String author, double price) {  
        this.bookId = id;  
        this.bookName = name;  
        this.author = author;  
        this.price = price;  
    }  
    public void setBookId(int id) {  
        bookId = id;  
    }  
    public void setBookName(String name) {  
        bookName = name;  
    }  
}
```

```
public void setAuthor(String a) {  
    author = a;  
}  
  
public void setPrice(double p) {  
    price = p;  
}  
  
public int getBookId() {  
    return bookId;  
}  
  
public String getBookName() {  
    return bookName;  
}  
  
public String getAuthor() {  
    return author;  
}  
  
public double getPrice() {  
    return price;  
}  
  
public void display() {  
    System.out.println("Book ID : " + bookId);  
    System.out.println("Book Name : " + bookName);  
    System.out.println("Author : " + author);  
    System.out.println("Price : " + price);  
}  
}
```

```
class Person {  
    String name = "sara";  
    int age = 12;  
}  
  
class Student extends Person {  
    int rollNo = 125;  
    void displayStudent() {  
        System.out.println("\nStudent Details");  
        System.out.println("Name : " + name);  
        System.out.println("Age : " + age);  
        System.out.println("Roll No : " + rollNo);  
    }  
}  
  
class Faculty extends Person {  
    String subject = "Java";  
    void displayFaculty() {  
        System.out.println("\nFaculty Details");  
        System.out.println("Name : " + name);  
        System.out.println("Age : " + age);  
        System.out.println("Subject : " + subject);  
    }  
}  
  
class Area {  
    // Method Overloading  
    void area(int side) {
```

```
        System.out.println("\nArea of Square = " + (side * side));  
    }
```

```
void area(int length, int breadth) {  
    System.out.println("Area of Rectangle = " + (length * breadth));  
}
```

```
void area(double radius) {  
    System.out.println("Area of Circle = " + (3.14 * radius * radius));  
}  
}
```

// Method Overriding

```
class Vehicle {  
    void display() {  
        System.out.println("\nThis is a Vehicle");  
    }  
}
```

```
class Car extends Vehicle {  
    void display() {  
        System.out.println("This is a Car");  
    }  
}
```

```
class Bike extends Vehicle {  
    void display() {  
        System.out.println("This is a Bike");  
    }  
}
```

```
    }  
}  
abstract class Shape {  
    abstract void draw();  
}  
class Circle extends Shape {  
    void draw() {  
        System.out.println("\nDrawing Circle");  
    }  
}  
class Rectangle extends Shape {  
    void draw() {  
        System.out.println("Drawing Rectangle");  
    }  
}  
interface Printable {  
    void print();  
}  
  
class Report implements Printable {  
    public void print() {  
        System.out.println("\nPrinting Library Report");  
    }  
}  
public class LibraryManagementSystem {
```

```
public static void main(String[] args) {

    Book b1 = new Book();
    b1.setBookId(101);
    b1.setBookName("Java Programming");
    b1.setAuthor("James Gosling");
    b1.setPrice(550);
    System.out.println("\nBook Details");
    b1.display();

    Book b2 = new Book(102, "Python", "Guido", 700);
    System.out.println("\nParameterized Constructor");
    b2.display();

    Student s = new Student();
    s.displayStudent();

    Faculty f = new Faculty();
    f.displayFaculty();

    // Method Overloading
    Area a = new Area();
    a.area(5);
    a.area(10, 20);
    a.area(4.5);

    // Method Overriding
    Vehicle v;
```

```
v = new Car();  
v.display();  
v = new Bike();  
v.display();  
Shape sh;  
sh = new Circle();  
sh.draw();  
sh = new Rectangle();  
sh.draw();  
Report r = new Report();  
r.print();  
}  
}
```

OUTPUT SCREENSHOT (SAMPLE RUN) :

Default Constructor Called

Book Details

Book ID : 101

Book Name : Java Programming

Author : James Gosling

Price : 550.0

Parameterized Constructor

Book ID : 102

Book Name : Python

Author : Guido

Price : 700.0

Student Details

Name : sara

Age : 12

Roll No : 125

Faculty Details

Name : sara

Age : 12

Subject : Java

Area of Square = 25

Area of Rectangle = 200

Area of Circle = 63.585

This is a Car

This is a Bike

Drawing Circle

Drawing Rectangle

Printing Library Report